

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

KHOA CÔNG NGHỆ THÔNG TIN



Hồ Nguyễn An Khang – 52200020

HỆ THỐNG CHẤM BÀI TRỰC TUYẾN - TDTUOJ

KHÓA LUẬN TỐT NGHIỆP

Kỹ thuật phần mềm

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

KHOA CÔNG NGHỆ THÔNG TIN



Hồ Nguyễn An Khang – 52200020

HỆ THỐNG CHẤM BÀI TRỰC TUYẾN - TDTUOJ

KHÓA LUẬN TỐT NGHIỆP

Kỹ thuật phần mềm

Người hướng dẫn:

ThS. Doãn Xuân Thanh

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Em xin chân thành cảm ơn quý thầy cô Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng đã tận tình giảng dạy và trang bị cho em những kiến thức nền tảng vững chắc trong suốt quá trình học tập. Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến giảng viên hướng dẫn **ThS. Doãn Xuân Thanh** đã dành thời gian định hướng, góp ý và hỗ trợ em trong từng giai đoạn thực hiện đề tài này. Bên cạnh đó, em xin cảm ơn gia đình và bạn bè đã luôn động viên, tạo điều kiện để em có thể hoàn thành tốt dự án.

TP. Hồ Chí Minh, ngày ... tháng ... năm 20...

Tác giả

(Ký tên và ghi rõ họ tên)

LỜI CAM ĐOAN

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của **ThS. Doãn Xuân Thanh**. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày ... tháng ... năm 20...

Tác giả

(Ký tên và ghi rõ họ tên)

HỆ THỐNG CHẤM BÀI TRỰC TUYẾN - TDTUOJ

TÓM TẮT

Dự án này trình bày quá trình nghiên cứu, thiết kế và triển khai ban đầu của Hệ thống Chấm bài Trực tuyến TDTUOJ – một nền tảng được xây dựng nhằm tự động hóa quy trình chấm bài lập trình và hỗ trợ tổ chức thi cử cho sinh viên Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng.

Hệ thống được thiết kế để phục vụ ba nhóm người dùng chính gồm sinh viên, giảng viên và quản trị viên, với các chức năng cốt lõi bao gồm nộp và chấm bài tự động cho nhiều ngôn ngữ lập trình, quản lý cuộc thi và bài toán, và phân tích hỗ trợ học tập thông minh bằng Large Language Model – Mô hình ngôn ngữ lớn (LLM). Kiến trúc hệ thống được xây dựng theo mô hình hai lớp tách biệt rõ ràng giữa backend và frontend, tích hợp với ba dịch vụ bên ngoài gồm Judging Service (Judge0), LLM và AI Parser Service.

Đến thời điểm báo cáo giữa kỳ, hệ thống đã hoàn thành phần lớn các chức năng cốt lõi. Phía backend được xây dựng bằng Java với Spring Boot, đã hiện thực hóa đầy đủ các Application Programming Interface (API) quản lý người dùng, quản lý bài toán và luồng xử lý bài nộp tích hợp Judge0. Phía frontend được xây dựng bằng React, đã hoàn thiện giao diện xem chi tiết bài toán theo slug, quản lý người dùng, tính năng nộp bài và hỗ trợ học tập bằng LLM.

ONLINE JUDGE SYSTEM - TDTUOJ

ABSTRACT

This project presents the research, design, and initial implementation of the TDTUOJ Online Judge System – a platform built to automate programming submission grading and support examination organization for students of the Faculty of Information Technology at Ton Duc Thang University.

The system is designed to serve three main user groups: students, lecturers, and administrators, with core features including automatic submission judging for multiple programming languages via Judge0, contest and problem management, and AI-powered learning support via LLM integration. The system architecture follows a clear two-layer separation between a Java Spring Boot backend and a React frontend, integrated with three external services: Judging Service (Judge0), LLM, and AI Parser Service.

As of the midterm report, the majority of core features have been implemented. The backend provides complete API coverage for user management, problem management, and Judge0-integrated submission processing. The frontend has completed the problem detail view by slug, user management interface, submission functionality, and LLM-powered hint and code explanation features.

MỤC LỤC

LỜI CẢM ƠN

LỜI CAM ĐOAN

TÓM TẮT

ABSTRACT

DANH MỤC HÌNH VẼ **v**

DANH MỤC BẢNG BIỂU **vi**

DANH MỤC CÁC CHỮ VIẾT TẮT **vii**

1	Mở đầu và tổng quan đề tài	1
1.1	Đặt vấn đề	1
1.2	Mục tiêu của hệ thống	1
1.3	Phạm vi đề tài	2
2	Công nghệ sử dụng	3
2.1	Backend – Java và Spring Boot	3
2.1.1	Tổng quan về Spring Boot	3
2.1.2	Kiến trúc MVC trong TDTUOJ	4
2.1.3	Bảo mật với Spring Security và JWT	4
2.2	Cơ sở dữ liệu – PostgreSQL	5
2.2.1	Tổng quan về PostgreSQL	5
2.2.2	Cấu hình kết nối	5
2.3	Docker và Containerization	6
2.3.1	Tổng quan về Docker	6

2.3.2	Containerization Judge0 với Docker	6
2.3.3	Docker Compose cho toàn hệ thống	8
2.4	Redis – Hàng đợi và Bộ nhớ đệm	8
2.4.1	Tổng quan về Redis	8
2.4.2	Hàng đợi bài nộp với Redis	9
2.4.3	Bộ nhớ đệm bằng xếp hạng	10
2.5	Judging Service – Judge0	10
2.5.1	Tổng quan về Judge0	10
2.5.2	Luồng xử lý bài nộp	11
2.6	Frontend – React	11
2.6.1	Tổng quan về React	11
2.6.2	Cấu trúc ứng dụng React trong TDTUOJ	12
2.7	Lưu trữ file – AWS S3	12
2.8	Tích hợp LLM	12
3	Phân tích hệ thống	14
3.1	Kiến trúc tổng quan	14
3.2	Sơ đồ Use Case tổng quan	14
3.3	Các tác nhân trong hệ thống	15
3.3.1	User (Người dùng)	15
3.3.2	Contest Creator (Người tạo cuộc thi)	16
3.3.3	Admin (Quản trị viên)	16
3.3.4	Các hệ thống bên ngoài	16
3.4	Thiết kế cơ sở dữ liệu	17
3.4.1	Sơ đồ thực thể – quan hệ	17
3.4.2	Nhóm bảng quản lý người dùng	18
3.4.3	Nhóm bảng quản lý bài toán	19
3.4.4	Nhóm bảng quản lý cuộc thi	19
3.4.5	Nhóm bảng quản lý bài nộp	20

4	Kết quả đã thực hiện	21
4.1	Backend – Java Spring Boot	21
4.1.1	Quản lý người dùng	21
4.1.2	Quản lý bài toán	21
4.1.3	Trực quan hóa thuật toán	21
4.1.4	Xử lý bài nộp	22
4.2	Tính năng hỗ trợ học tập bằng LLM	22
4.2.1	Phân tích độ phức tạp code	22
4.2.2	Giải thích code và lỗi	22
4.2.3	Gợi ý hướng giải quyết bài toán	23
4.3	Frontend – React	23
4.3.1	Giao diện xem bài toán	23
4.3.2	Giao diện quản lý người dùng	23
4.3.3	Tính năng nộp bài	23
5	Hướng phát triển	24
5.1	Các tính năng còn lại cần hoàn thiện	24
5.2	Kế hoạch hoàn thiện	24
5.2.1	Hoàn thiện chức năng cuộc thi và tổ chức	24
5.2.2	Trực quan hóa thuật toán	24
5.2.3	Tích hợp trình gỡ lỗi	25
5.2.4	Tích hợp AI và hoàn thiện các dịch vụ	25
5.2.5	Hoàn thiện và đồng nhất giao diện người dùng	25
5.2.6	Kiểm thử và triển khai	25
6	Kết luận	27
6.1	Tổng kết	27
6.2	Ý nghĩa của đề tài	27

DANH MỤC HÌNH VẼ

3.1	Sơ đồ Use Case tổng quan của hệ thống TDTUOJ	15
3.2	Sơ đồ thực thể – quan hệ của hệ thống TDTUOJ	17

DANH MỤC BẢNG BIỂU

DANH MỤC CÁC CHỮ VIẾT TẮT

AC	Accepted
AI	Artificial Intelligence – Trí tuệ nhân tạo
API	Application Programming Interface
CE	Compilation Error
CNTT	Công nghệ Thông tin
CSS	Cascading Style Sheets
ERD	Entity-Relationship Diagram – Sơ đồ thực thể quan hệ
HTTP	Hypertext Transfer Protocol
JPA	Java Persistence API
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model – Mô hình ngôn ngữ lớn
MLE	Memory Limit Exceeded
MVC	Model-View-Controller
OJ	Online Judge
PDF	Portable Document Format
PK	Primary Key – Khóa chính
FK	Foreign Key – Khóa ngoại
RE	Runtime Error

REST	Representational State Transfer
TDTU	Trường Đại học Tôn Đức Thắng
TLE	Time Limit Exceeded
UI	User Interface
UX	User Experience
WA	Wrong Answer

CHƯƠNG 1. MỞ ĐẦU VÀ TỔNG QUAN ĐỀ TÀI

1.1 Đặt vấn đề

Hiện nay, việc tổ chức các kỳ thi lập trình và rèn luyện kỹ năng giải thuật cho sinh viên tại Trường Đại học Tôn Đức Thắng vẫn còn nhiều hạn chế về mặt công cụ hỗ trợ. Chấm bài thủ công tiêu tốn nhiều thời gian và công sức của giảng viên, đồng thời khó đảm bảo tính nhất quán khi số lượng sinh viên ngày càng tăng. Bên cạnh đó, sinh viên thiếu một môi trường luyện tập chủ động với phản hồi tức thì để tự nâng cao kỹ năng lập trình ngoài giờ học.

Để giải quyết những vấn đề này, đề tài đề xuất xây dựng Hệ thống Chấm bài Trực tuyến TDTUOJ – một nền tảng tự động hóa toàn bộ quy trình chấm bài, hỗ trợ tổ chức thi cử quy mô lớn và tạo ra môi trường luyện tập lập trình chuyên nghiệp trong khuôn khổ học thuật của nhà trường.

1.2 Mục tiêu của hệ thống

TDTUOJ được xây dựng nhằm đạt một tập hợp các mục tiêu cụ thể và có tính ứng dụng cao. Về mặt kỹ thuật, hệ thống hỗ trợ đa ngôn ngữ lập trình, cho phép người dùng nộp bài viết bằng C, C++, Java và Python, đáp ứng nhu cầu đa dạng của sinh viên ở nhiều trình độ và môn học khác nhau. Song song với đó, hệ thống cung cấp đầy đủ công cụ để quản lý cuộc thi và bài tập, bao gồm tổ chức contest, quản lý danh sách bài toán, chấm điểm tự động và xếp hạng người tham gia theo thời gian thực.

Ngoài các chức năng cốt lõi, hệ thống tích hợp một số tính năng nâng cao phục vụ học tập. Tính năng phân tích mã nguồn bằng LLM cho phép giải thích code, gợi ý hướng giải và phân tích độ phức tạp thuật toán ngay trên giao diện – giúp sinh viên hiểu rõ hơn về hiệu suất và logic của giải pháp mình đề xuất. Ngoài ra, hệ thống tích hợp AI Parser Service để tự động đọc đề bài từ file Portable Document Format (PDF) và chuyển đổi thành dữ liệu có cấu trúc, giúp giảng viên tiết kiệm đáng kể thời gian soạn thảo.

1.3 Phạm vi đề tài

Đề tài tập trung xây dựng một hệ thống hoàn chỉnh phục vụ sinh viên và giảng viên Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng. Hệ thống quản lý ba vai trò với phân quyền rõ ràng: User (người dùng thông thường), Contest Creator (người tổ chức và quản lý cuộc thi), và Admin (quản trị viên với toàn quyền điều hành). Về mặt kiến trúc, phạm vi đề tài bao gồm phát triển một backend Representational State Transfer (REST)ful API bằng Java Spring Boot, một giao diện người dùng bằng React, và tích hợp ba dịch vụ bên ngoài: Judging Service (Judge0), LLM và AI Parser Service.

CHƯƠNG 2. CÔNG NGHỆ SỬ DỤNG

Hệ thống TDTUOJ được xây dựng trên nền tảng các công nghệ hiện đại, được lựa chọn có chủ đích để đảm bảo hiệu năng, khả năng mở rộng và dễ dàng bảo trì. Chương này trình bày chi tiết từng công nghệ được áp dụng, lý do lựa chọn và cách thức tích hợp vào hệ thống.

2.1 Backend – Java và Spring Boot

2.1.1 Tổng quan về Spring Boot

Spring Boot là một framework mã nguồn mở được phát triển bởi Pivotal (nay là VMware), xây dựng trên nền tảng Spring Framework của Java. Spring Boot ra đời nhằm giải quyết một trong những điểm yếu lịch sử của Spring Framework truyền thống – đó là sự phức tạp trong cấu hình. Thay vì yêu cầu lập trình viên tự cấu hình hàng trăm dòng XML hoặc annotation, Spring Boot áp dụng triết lý *convention over configuration* (ưu tiên quy ước hơn cấu hình), tự động phát hiện và cấu hình các thành phần dựa trên các thư viện có mặt trong classpath.

Cơ chế trung tâm của Spring Boot là *Auto-configuration* – khi thêm dependency `spring-boot-starter-web` vào dự án, Spring Boot tự động khởi tạo một máy chủ Tomcat nhúng, cấu hình `DispatcherServlet` và thiết lập Jackson để xử lý JSON mà không cần bất kỳ cấu hình thủ công nào. Điều này giúp lập trình viên tập trung vào logic nghiệp vụ thay vì quản lý cơ sở hạ tầng.

Spring Boot cũng cung cấp hệ sinh thái *Starter dependencies* phong phú, mỗi starter đóng gói một tập hợp các thư viện liên quan: `spring-boot-starter-data-jpa` tích hợp Hibernate và Spring Data JPA, `spring-boot-starter-security` cung cấp toàn bộ khung bảo mật, và `spring-boot-starter-test` tích hợp JUnit, Mockito và các công cụ kiểm thử khác.

2.1.2 Kiến trúc MVC trong TDTUOJ

TDTUOJ áp dụng kiến trúc Model-View-Controller (MVC) kết hợp với mô hình RESTful API. Trong Spring Boot, kiến trúc này được tổ chức thành các lớp tách biệt rõ ràng:

Lớp **Controller** đảm nhận vai trò tiếp nhận các Hypertext Transfer Protocol (HTTP) request từ frontend. Mỗi controller được đánh dấu bằng annotation `@RestController` và định nghĩa các endpoint thông qua `@GetMapping`, `@PostMapping`, `@PutMapping` và `@DeleteMapping`. Ví dụ, `ProblemController` xử lý toàn bộ các request liên quan đến bài toán, từ lấy danh sách, xem chi tiết theo slug đến tạo mới và cập nhật.

Lớp **Service** chứa toàn bộ logic nghiệp vụ, được đánh dấu bằng `@Service`. Đây là nơi thực thi các quy tắc kinh doanh như kiểm tra quyền hạn người dùng, xử lý trạng thái bài nộp, tính điểm cuộc thi và gọi các dịch vụ bên ngoài.

Lớp **Repository** tương tác trực tiếp với cơ sở dữ liệu thông qua Spring Data Java Persistence API (JPA). Các interface kế thừa `JpaRepository` được Spring Boot tự động hiện thực, cung cấp sẵn các phương thức CRUD mà không cần viết câu lệnh SQL thủ công. Truy vấn tùy chỉnh có thể được định nghĩa bằng cú pháp `findBy...` hoặc annotation `@Query` với ngôn ngữ JPQL.

2.1.3 Bảo mật với Spring Security và JWT

Hệ thống TDTUOJ sử dụng JSON Web Token (JWT) để xác thực và phân quyền người dùng. Cơ chế hoạt động như sau: sau khi đăng nhập thành công, backend tạo ra một JWT chứa thông tin định danh người dùng và vai trò của họ, ký bằng khóa bí mật và trả về cho client. Với mỗi request tiếp theo, client đính kèm token trong header `Authorization: Bearer <token>`, backend kiểm tra tính hợp lệ của token trước khi xử lý request.

Spring Security được cấu hình với một `JwtAuthenticationFilter` tùy chỉnh, chặn mọi request đến và xác minh token trước khi cho phép đi qua. Phân quyền

được thực thi tại lớp controller thông qua annotation `@PreAuthorize`, cho phép kiểm soát chi tiết đến từng endpoint – chẳng hạn chỉ Admin mới có thể gọi các API quản lý tài khoản, trong khi Contest Creator mới có thể tạo cuộc thi.

2.2 Cơ sở dữ liệu – PostgreSQL

2.2.1 Tổng quan về PostgreSQL

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở mạnh mẽ, nổi tiếng với khả năng tuân thủ chuẩn SQL nghiêm ngặt, hỗ trợ các kiểu dữ liệu phong phú và độ ổn định cao trong môi trường production. So với MySQL, PostgreSQL có ưu thế trong việc xử lý các truy vấn phức tạp với nhiều phép JOIN, hỗ trợ các kiểu dữ liệu đặc biệt như JSONB, ARRAY và UUID, và có cơ chế kiểm soát đồng thời đa phiên bản (MVCC) hiệu quả hơn trong các tình huống ghi/đọc đồng thời – rất phù hợp với kịch bản nhiều người dùng cùng nộp bài trong một cuộc thi.

Trong TDTUOJ, PostgreSQL được chọn làm cơ sở dữ liệu chính, lưu trữ toàn bộ 20 bảng mô tả trong sơ đồ Entity-Relationship Diagram – Sơ đồ thực thể quan hệ (ERD). Spring Data JPA với Hibernate làm ORM layer đảm nhận việc ánh xạ giữa các entity Java và các bảng trong cơ sở dữ liệu.

2.2.2 Cấu hình kết nối

Kết nối từ Spring Boot đến PostgreSQL được cấu hình trong file `application.properties` với các thông số `spring.datasource.url`, `spring.datasource.username` và `spring.datasource.password`. Connection pooling được quản lý bởi HikariCP – pool kết nối mặc định và hiệu năng cao nhất của Spring Boot – giúp tái sử dụng kết nối và giảm độ trễ khi có nhiều request đồng thời.

2.3 Docker và Containerization

2.3.1 Tổng quan về Docker

Docker là nền tảng container hóa cho phép đóng gói ứng dụng cùng toàn bộ môi trường phụ thuộc (runtime, thư viện, biến môi trường, file cấu hình) vào một đơn vị gọi là *container*. Khác với máy ảo truyền thống phải ảo hóa toàn bộ hệ điều hành, container chia sẻ kernel của máy chủ, giúp khởi động nhanh hơn, tiêu tốn ít tài nguyên hơn và dễ dàng di chuyển giữa các môi trường.

2.3.2 Containerization Judge0 với Docker

Judge0 là dịch vụ chấm bài mã nguồn mở được triển khai thông qua Docker. Hệ thống Judge0 bao gồm nhiều thành phần phối hợp với nhau, được quản lý bằng Docker Compose:

- **judge0-server**: Container chính cung cấp RESTful API để nhận bài nộp và trả về kết quả.
- **judge0-workers**: Các worker container thực thi mã nguồn trong môi trường sandbox cô lập, sử dụng *isolate* – một công cụ kiểm soát tài nguyên dựa trên Linux cgroups và namespaces.
- **Redis**: Hàng đợi bài nộp, lưu trữ các submission đang chờ được xử lý bởi worker.
- **PostgreSQL**: Cơ sở dữ liệu nội bộ của Judge0, lưu kết quả và lịch sử chấm bài.

Cấu hình triển khai Judge0 bằng Docker Compose được mô tả như sau:

```
services:
  server:
    image: judge0/judge0:latest
    ports:
      - "2358:2358"
    environment:
```

```
    REDIS_HOST: redis
    POSTGRES_HOST: db
  depends_on:
    - redis
    - db

workers:
  image: judge0/judge0:latest
  command: ["./scripts/workers"]
  privileged: true
  depends_on:
    - server

redis:
  image: redis:7-alpine

db:
  image: postgres:16-alpine
  environment:
    POSTGRES_PASSWORD: judge0
```

Lưu ý quan trọng: container workers cần được chạy với quyền `privileged: true` vì *isolate* cần quyền truy cập kernel-level để tạo ra các cgroups và namespaces kiểm soát tài nguyên. Backend TDTUOJ giao tiếp với Judge0 thông qua API tại cổng 2358, gửi mã nguồn kèm theo ngôn ngữ lập trình và giới hạn thời gian/bộ nhớ lấy từ bảng `test_cases`.

2.3.3 Docker Compose cho toàn hệ thống

Ngoài Judge0, toàn bộ môi trường phát triển của TDTUOJ cũng được container hóa. File `docker-compose.yml` của dự án tổ chức các dịch vụ sau:

- **tdtuoj-backend:** Container chạy Spring Boot application, được build từ `Dockerfile` trong thư mục `TDTUOJ_backend`.
- **tdtuoj-postgres:** Container PostgreSQL phục vụ dữ liệu của TDTUOJ, độc lập với PostgreSQL của Judge0.
- **tdtuoj-frontend:** Container React application phục vụ giao diện người dùng.

Việc sử dụng Docker giúp đảm bảo tính nhất quán giữa môi trường phát triển của từng thành viên và môi trường production, đồng thời đơn giản hóa quá trình thiết lập – thay vì cài đặt Java, Node.js, PostgreSQL riêng lẻ, người phát triển chỉ cần chạy `docker-compose up` để khởi động toàn bộ hệ thống.

2.4 Redis – Hàng đợi và Bộ nhớ đệm

2.4.1 Tổng quan về Redis

Redis (Remote Dictionary Server) là một kho lưu trữ dữ liệu trong bộ nhớ (in-memory data store) mã nguồn mở, thường được sử dụng như một cơ sở dữ liệu tốc độ cao, bộ nhớ đệm (cache) và message broker. Điểm khác biệt cơ bản của Redis so với các hệ quản trị cơ sở dữ liệu truyền thống là toàn bộ dữ liệu được lưu trong RAM thay vì đĩa cứng, cho phép đạt tốc độ đọc/ghi cực cao – thường đạt hàng trăm nghìn thao tác mỗi giây. Redis hỗ trợ nhiều cấu trúc dữ liệu phong phú như string, hash, list, set, sorted set và stream, mỗi loại phù hợp với một bài toán cụ thể.

Trong TDTUOJ, Redis được sử dụng theo hai mục đích chính: làm hàng đợi bài nộp (submission queue) để xử lý bất đồng bộ với Judge0, và làm bộ nhớ đệm cho bảng xếp hạng (leaderboard cache) trong các cuộc thi.

2.4.2 Hàng đợi bài nộp với Redis

Khi nhiều người dùng nộp bài cùng lúc – đặc biệt trong các cuộc thi có hàng trăm thí sinh – hệ thống phải xử lý một lượng lớn submission đồng thời. Nếu backend gửi trực tiếp từng submission đến Judge0 và chờ kết quả ngay lập tức (synchronous), các request sẽ chiếm giữ thread trong suốt thời gian Judge0 thực thi mã nguồn, dẫn đến hiện tượng thread pool cạn kiệt và hệ thống trở nên không phản hồi.

Để giải quyết vấn đề này, TDTUOJ áp dụng mô hình hàng đợi bất đồng bộ với Redis theo luồng sau:

Khi người dùng nộp bài, backend không gọi Judge0 trực tiếp mà thay vào đó tạo một bản ghi submission với trạng thái PENDING trong cơ sở dữ liệu, sau đó đẩy một message chứa `submission_id` vào một Redis List (đóng vai trò hàng đợi) và lập tức trả về phản hồi cho người dùng. Người dùng nhận được xác nhận bài đã được tiếp nhận mà không phải chờ kết quả chấm.

Ở phía tiêu thụ hàng đợi, một hoặc nhiều *submission worker* chạy nền liên tục lắng nghe hàng đợi Redis bằng lệnh BLPOP (blocking left pop) – lệnh này chặn worker cho đến khi có message mới xuất hiện trong hàng đợi, tránh việc polling liên tục tiêu tốn CPU. Khi nhận được `submission_id`, worker truy xuất thông tin submission và test cases từ cơ sở dữ liệu, gửi lần lượt từng test case đến Judge0 API, nhận về kết quả, tổng hợp verdict cuối cùng và cập nhật bản ghi trong bảng submissions từ trạng thái PENDING sang JUDGED.

Luồng này có thể được mô tả qua các bước tuần tự sau:

1. Frontend gửi POST `/submissions` đến backend.
2. Backend lưu submission với trạng thái PENDING, đẩy `submission_id` vào Redis List `submission_queue`.
3. Backend trả về HTTP 202 Accepted cùng `submission_id` cho frontend.
4. Frontend polling endpoint GET `/submissions/{id}` theo chu kỳ để kiểm tra trạng thái.

5. Worker nhận `submission_id` từ hàng đợi, gửi đến Judge0, nhận verdict và cập nhật cơ sở dữ liệu.
6. Lần polling tiếp theo của frontend nhận được kết quả đã hoàn chỉnh.

Mô hình này mang lại hai lợi ích quan trọng: thứ nhất, hệ thống có khả năng mở rộng theo chiều ngang bằng cách tăng số lượng worker xử lý song song khi tải cao; thứ hai, các submission không bao giờ bị mất – nếu một worker gặp sự cố trong khi xử lý, message vẫn có thể được xử lý lại thay vì bị bỏ qua hoàn toàn.

2.4.3 Bộ nhớ đệm bảng xếp hạng

Trong thời gian diễn ra cuộc thi, bảng xếp hạng cần được cập nhật liên tục và phục vụ hàng trăm request xem leaderboard đồng thời. Nếu mỗi request đều thực hiện truy vấn tổng hợp phức tạp trên bảng `contest_participations` – tính điểm, sắp xếp, phân trang – thì cơ sở dữ liệu sẽ chịu tải rất lớn và thời gian phản hồi sẽ tăng cao.

TDTUOJ giải quyết vấn đề này bằng cách sử dụng Redis làm lớp cache cho bảng xếp hạng, phản ánh thiết kế của bảng `leaderboard_cache` trong sơ đồ ERD. Sau mỗi lần có bài nộp được chấp nhận trong cuộc thi, hệ thống tính toán lại thứ hạng và lưu kết quả đã được serialize vào Redis với một TTL (time-to-live) ngắn. Các request xem bảng xếp hạng tiếp theo đọc trực tiếp từ Redis thay vì truy vấn cơ sở dữ liệu, giúp thời gian phản hồi giảm từ hàng trăm millisecond xuống còn dưới một millisecond. Khi TTL hết hạn hoặc có cập nhật mới, cache được làm mới tự động, đảm bảo tính nhất quán giữa dữ liệu hiển thị và trạng thái thực tế của cuộc thi.

2.5 Judging Service – Judge0

2.5.1 Tổng quan về Judge0

Judge0 là một dịch vụ chấm bài mã nguồn mở, cung cấp môi trường thực thi mã nguồn an toàn và cô lập hỗ trợ hơn 60 ngôn ngữ lập trình. Thay vì tự xây dựng cơ sở hạ tầng sandbox từ đầu – một bài toán kỹ thuật phức tạp đòi hỏi kiến thức sâu về bảo mật hệ điều hành – TDTUOJ tích hợp Judge0 như một dịch vụ độc lập có thể gọi

qua API.

2.5.2 Luồng xử lý bài nộp

Luồng tích hợp Judge0 trong TDTUOJ được thực thi theo các bước sau. Đầu tiên, khi người dùng nhấn nộp bài trên frontend, một HTTP POST request được gửi đến backend kèm theo mã nguồn, ngôn ngữ lập trình và thông tin bài toán. Backend đọc danh sách test cases từ bảng `test_cases` và gửi từng test case đến endpoint POST `/submissions` của Judge0, kèm theo `source_code`, `language_id`, `stdin` (đầu vào), `expected_output`, `cpu_time_limit` và `memory_limit`.

Judge0 nhận request, đưa vào hàng đợi Redis và phân công cho một worker xử lý. Worker thực thi mã nguồn trong môi trường sandbox cô lập, so sánh đầu ra với `expected_output` và trả về token định danh. Backend poll kết quả theo token đó, nhận về verdict tương ứng: Accepted (AC), Wrong Answer (WA), Time Limit Exceeded (TLE), Memory Limit Exceeded (MLE), Runtime Error (RE) hoặc Compilation Error (CE). Sau khi nhận đủ kết quả của tất cả test cases, backend tổng hợp verdict cuối cùng và lưu vào bảng `submissions`.

Toàn bộ luồng này được thiết kế bất đồng bộ ở phía backend: thread xử lý request không bị chặn trong khi chờ Judge0 – backend sử dụng cơ chế polling hoặc webhook để nhận kết quả, đảm bảo khả năng xử lý nhiều bài nộp song song mà không làm ảnh hưởng đến hiệu năng tổng thể.

2.6 Frontend – React

2.6.1 Tổng quan về React

React là thư viện JavaScript mã nguồn mở được Meta phát triển, chuyên dùng để xây dựng giao diện người dùng. Điểm cốt lõi của React là mô hình lập trình khai báo (declarative) dựa trên *component* – mỗi phần của giao diện được đóng gói thành một component độc lập với state và logic riêng, có thể tái sử dụng ở nhiều nơi trong ứng dụng.

React sử dụng *Virtual DOM* – một bản sao nhẹ của DOM thực trong bộ nhớ.

Khi state thay đổi, React tính toán sự khác biệt (diffing) giữa Virtual DOM cũ và mới, sau đó chỉ cập nhật đúng những phần thực sự thay đổi trên DOM thực. Cơ chế này giúp tối ưu hiệu năng đáng kể so với việc thao tác DOM trực tiếp.

2.6.2 Cấu trúc ứng dụng React trong TDTUOJ

Frontend của TDTUOJ được xây dựng theo mô hình SPA (*Single Page Application*) sử dụng React Router để điều hướng giữa các trang mà không cần tải lại toàn bộ trang. Cấu trúc thư mục được tổ chức theo tính năng: mỗi module lớn như quản lý bài toán, cuộc thi, người dùng và nộp bài được đặt trong thư mục riêng, chứa các component, hook tùy chỉnh và file gọi API tương ứng.

Giao tiếp với backend thực hiện qua HTTP request sử dụng Fetch API, với JWT được đính kèm trong header Authorization của mỗi request yêu cầu xác thực. Dữ liệu trao đổi theo định dạng JavaScript Object Notation (JSON), được React tự động cập nhật vào state component thông qua các hook `useState` và `useEffect`.

2.7 Lưu trữ file – AWS S3

Nội dung đề bài và dữ liệu test cases được lưu trữ trên **AWS S3** (Simple Storage Service) – dịch vụ lưu trữ đối tượng có khả năng mở rộng không giới hạn của Amazon. Thay vì lưu trực tiếp trong cơ sở dữ liệu, các file lớn như đề bài định dạng PDF và file input/output của test cases được upload lên S3 bucket, còn cơ sở dữ liệu chỉ lưu đường dẫn URL trỏ đến các file đó – thể hiện qua các trường `statement_file_url`, `input_file_url` và `expected_output_file_url` trong thiết kế bảng. Cách tiếp cận này giúp cơ sở dữ liệu không bị phình to, đồng thời tận dụng được khả năng phân phối nội dung toàn cầu và độ bền lưu trữ cao của S3.

2.8 Tích hợp LLM

Hệ thống tích hợp LLM thông qua API để cung cấp ba tính năng hỗ trợ học tập: phân tích độ phức tạp thuật toán, giải thích mã nguồn và lỗi, và gợi ý hướng giải quyết bài toán. Backend đóng vai trò trung gian – tiếp nhận yêu cầu từ frontend, xây

dựng prompt phù hợp chứa ngữ cảnh bài toán và mã nguồn của người dùng, gửi đến LLM API và trả kết quả về frontend để hiển thị. Kiến trúc này tách biệt hoàn toàn việc gọi LLM khỏi frontend, giúp bảo vệ API key và cho phép thay đổi nhà cung cấp LLM mà không ảnh hưởng đến giao diện.

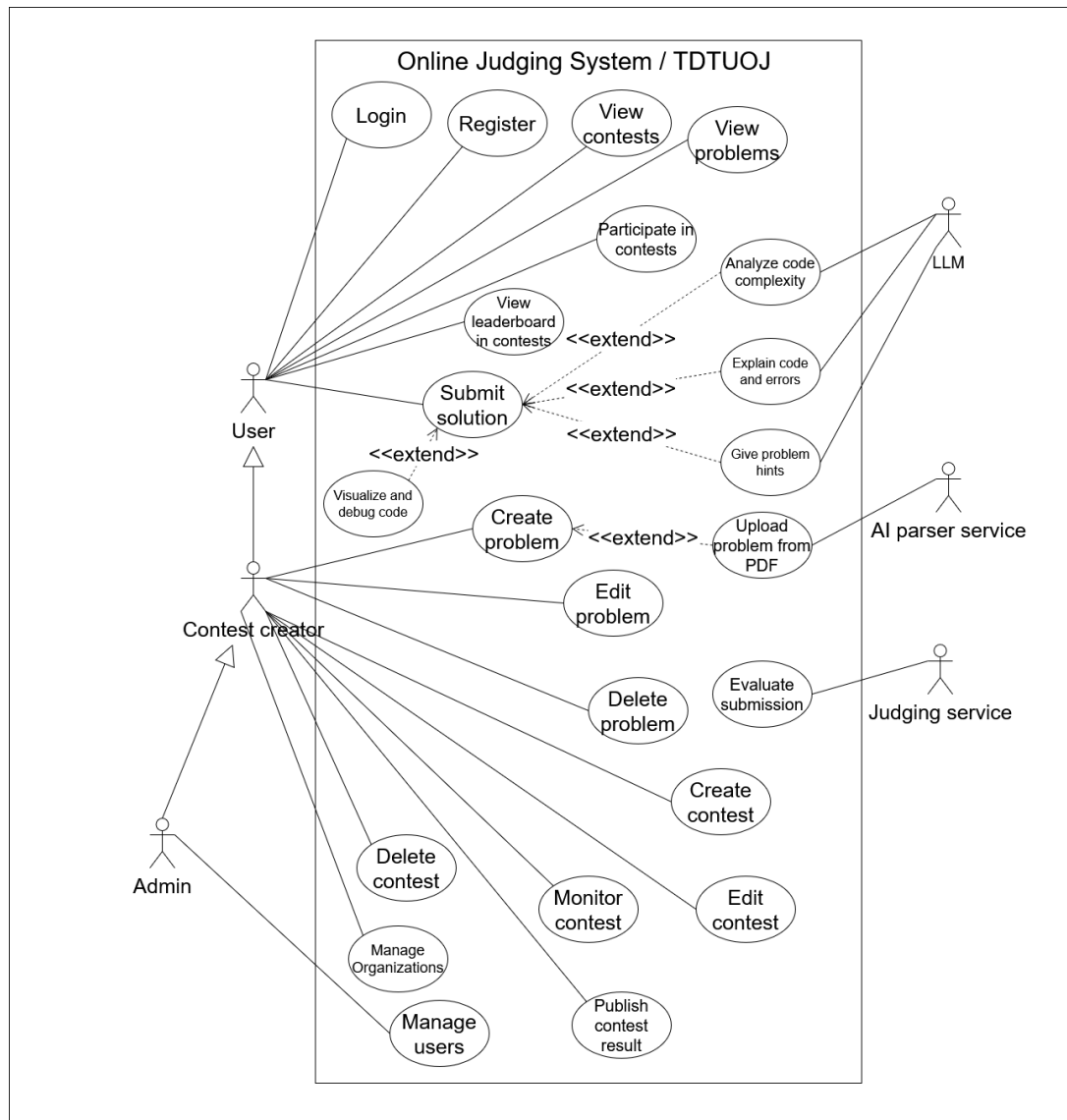
CHƯƠNG 3. PHÂN TÍCH HỆ THỐNG

3.1 Kiến trúc tổng quan

TDTUOJ được xây dựng theo kiến trúc tách biệt hoàn toàn giữa backend và frontend, trong đó backend đóng vai trò máy chủ cung cấp RESTful API và frontend là ứng dụng phía client tiêu thụ các API đó. Ngoài hai thành phần chính, hệ thống tích hợp với ba dịch vụ bên ngoài – Judging Service (Judge0), LLM và AI Parser Service – mỗi dịch vụ được tách biệt độc lập để đảm bảo tính ổn định và khả năng mở rộng.

3.2 Sơ đồ Use Case tổng quan

Sơ đồ Use Case của TDTUOJ mô tả đầy đủ mối quan hệ giữa ba nhóm tác nhân người dùng và ba hệ thống bên ngoài, thể hiện rõ phạm vi chức năng của từng vai trò. Như minh họa trong Hình 3.1, hệ thống bao gồm User, Contest Creator và Admin theo quan hệ kế thừa, cùng với các tác nhân bên ngoài là LLM, AI Parser Service và Judging Service.



Hình 3.1: Sơ đồ Use Case tổng quan của hệ thống TDTUOJ

3.3 Các tác nhân trong hệ thống

3.3.1 User (Người dùng)

User là sinh viên hoặc người tham gia sử dụng hệ thống để luyện tập và tham gia cuộc thi. Đây là vai trò cơ bản nhất và là nền tảng của quan hệ kế thừa – Contest Creator và Admin đều kế thừa toàn bộ quyền hạn của vai trò này. Theo sơ đồ Use Case, User có thể đăng nhập, đăng ký tài khoản, xem danh sách cuộc thi, xem danh sách bài toán, tham gia cuộc thi và xem bảng xếp hạng. Ngoài ra, User có thể nộp bài giải bằng C, C++, Java hoặc Python thông qua chức năng Submit solution, từ đó kích

hoạt ba tính năng mở rộng: phân tích độ phức tạp code, giải thích code và lỗi, và gợi ý hướng giải – tất cả được cung cấp bởi tác nhân ngoài LLM. Chức năng Visualize and debug code cũng được liên kết với Submit solution như một tính năng hỗ trợ bổ sung.

3.3.2 Contest Creator (Người tạo cuộc thi)

Contest Creator là giảng viên hoặc người được cấp quyền tổ chức cuộc thi và quản lý bài toán. Ngoài toàn bộ quyền hạn của User, Contest Creator có thể tạo bài toán mới kèm bộ test cases, chỉnh sửa và xóa bài toán đã có. Chức năng Create problem có thể mở rộng bằng tính năng Upload problem from PDF – thực hiện qua AI Parser Service, tự động chuyển đổi đề bài từ file PDF thành dữ liệu có cấu trúc. Về tổ chức thi, Contest Creator có quyền tạo, chỉnh sửa, xóa, giám sát và công bố kết quả cuộc thi. Việc chấm bài được xử lý hoàn toàn tự động bởi Judging Service (Judge0).

3.3.3 Admin (Quản trị viên)

Admin là vai trò có quyền quản trị cao nhất trong hệ thống, kế thừa toàn bộ quyền hạn của Contest Creator. Ngoài ra, Admin có thêm hai chức năng quản trị toàn hệ thống: Manage users – xem danh sách, chỉnh sửa thông tin, kích hoạt và vô hiệu hóa tài khoản – và Manage Organizations, cho phép tạo và quản lý các tổ chức đại diện cho từng khoa hoặc lớp học.

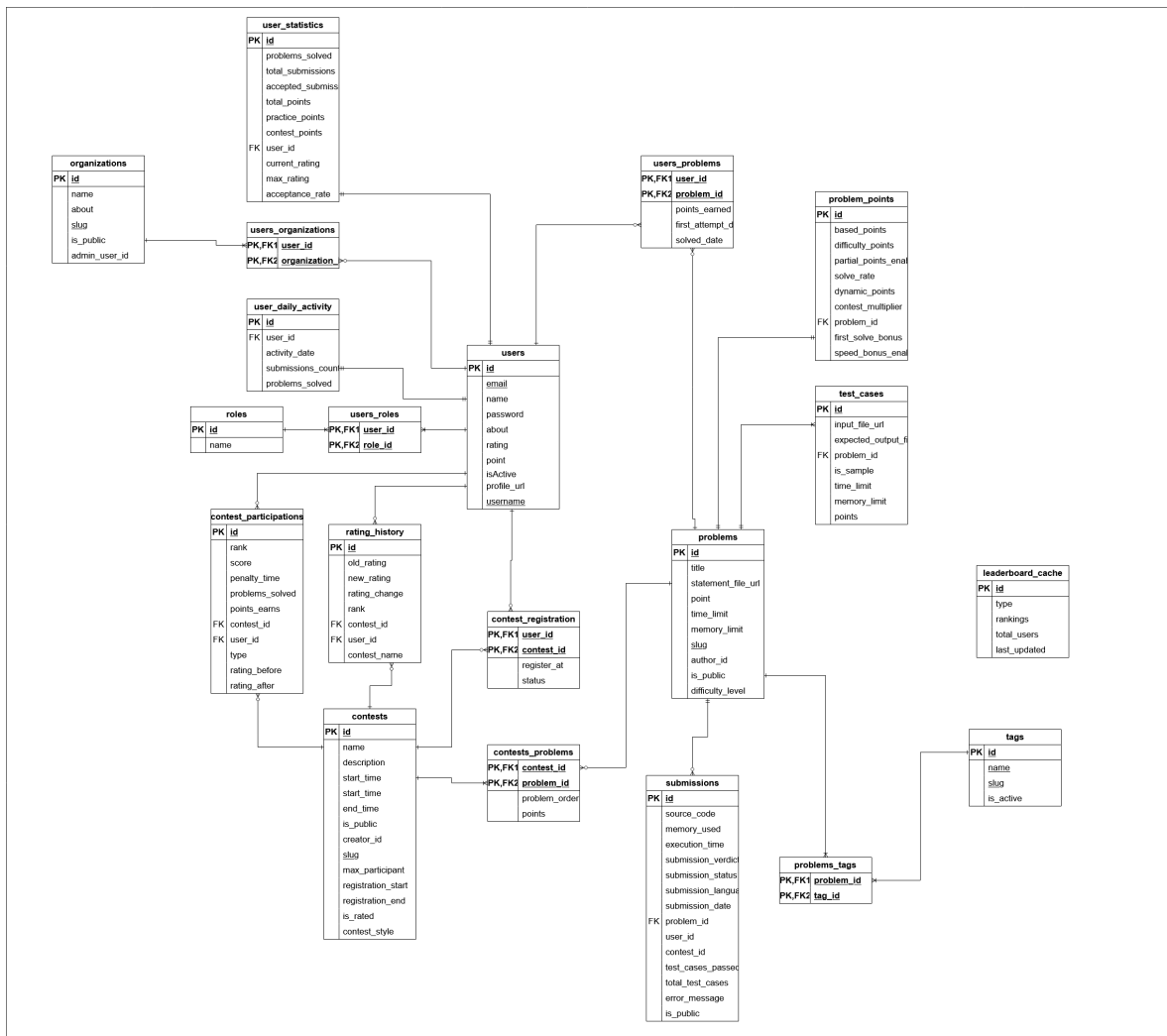
3.3.4 Các hệ thống bên ngoài

TDTUOJ tích hợp với ba dịch vụ bên ngoài chuyên biệt. LLM cung cấp ba tính năng hỗ trợ học tập được kích hoạt sau khi người dùng nộp bài: phân tích độ phức tạp code, giải thích code và lỗi, và gợi ý hướng giải. AI Parser Service trích xuất và chuyển đổi nội dung đề bài từ file PDF thành dữ liệu có cấu trúc khi Contest Creator tạo bài toán mới. Judging Service thực thi và chấm điểm bài nộp thông qua Judge0, trả về verdict tương ứng.

3.4 Thiết kế cơ sở dữ liệu

3.4.1 Sơ đồ thực thể – quan hệ

Sơ đồ ERD trong Hình 3.2 thể hiện toàn bộ cấu trúc cơ sở dữ liệu của TDTUOJ với 20 bảng được tổ chức thành bốn nhóm chức năng: quản lý người dùng, quản lý bài toán, quản lý cuộc thi và quản lý bài nộp. Thiết kế đảm bảo tính toàn vẹn dữ liệu thông qua các ràng buộc khóa ngoại (Foreign Key – Khóa ngoại (FK)) và hỗ trợ hiệu quả các truy vấn phức tạp trong quá trình vận hành.



Hình 3.2: Sơ đồ thực thể – quan hệ của hệ thống TDTUOJ

3.4.2 Nhóm bảng quản lý người dùng

Nhóm bảng này xử lý toàn bộ dữ liệu liên quan đến tài khoản và hoạt động người dùng trong hệ thống.

Bảng `users` là bảng trung tâm của toàn hệ thống, lưu thông tin cơ bản của mỗi tài khoản gồm `email`, `name`, `password`, `username`, `rating`, `point`, `isActive` và `profile_url`. Các trường `rating` và `point` phản ánh năng lực và thành tích tích lũy qua quá trình tham gia cuộc thi và luyện tập.

Bảng `roles` và bảng liên kết `users_roles` hiện thực cơ chế phân quyền nhiều vai trò. Một người dùng có thể giữ nhiều vai trò đồng thời, mỗi vai trò được định nghĩa bằng trường `name`. Quan hệ nhiều-nhiều này được quản lý qua `users_roles` với hai khóa ngoại `user_id` và `role_id`.

Bảng `organizations` lưu thông tin các tổ chức trong hệ thống với các trường `name`, `about`, `slug`, `is_public` và `admin_user_id`. Quan hệ nhiều-nhiều giữa người dùng và tổ chức được quản lý qua bảng liên kết `users_organizations`.

Bảng `user_statistics` theo dõi các chỉ số tổng hợp của mỗi người dùng, bao gồm `problems_solved`, `total_submissions`, `accepted_submissions`, `total_points`, `practice_points`, `contest_points`, `current_rating`, `max_rating` và `acceptance_rate`. Bảng này phục vụ chủ yếu cho các màn hình thống kê và bảng xếp hạng.

Bảng `user_daily_activity` ghi nhận hoạt động hàng ngày của người dùng qua các trường `activity_date`, `submissions_count` và `problems_solved`, phục vụ hiển thị biểu đồ hoạt động theo ngày tương tự GitHub contribution graph.

Bảng `rating_history` lưu lịch sử thay đổi `rating` qua từng cuộc thi, bao gồm `old_rating`, `new_rating`, `rating_change`, `rank`, `contest_id`, `user_id` và `contest_name`, giúp người dùng theo dõi sự tiến bộ theo thời gian.

3.4.3 Nhóm bảng quản lý bài toán

Nhóm bảng này quản lý nội dung bài toán, test cases, cấu trúc tính điểm và phân loại theo nhãn.

Bảng `problems` lưu thông tin chính của mỗi bài toán với các trường `title`, `statement_file_url`, `point`, `time_limit`, `memory_limit`, `slug`, `author_id`, `is_public` và `difficulty_level`. Nội dung đề bài được lưu dạng file trên S3 bucket của AWS và trỏ đến qua `statement_file_url`, trong khi `slug` được dùng để tạo URL thân thiện cho từng bài toán.

Bảng `test_cases` lưu các bộ dữ liệu kiểm thử gồm `input_file_url`, `expected_output_file_url`, `is_sample`, `time_limit`, `memory_limit` và `points`. Trường `is_sample` phân biệt test case mẫu hiển thị công khai với test case ẩn chỉ dùng để chấm bài. Các trường `input_file_url` và `expected_output_file_url` lưu đường dẫn tới file chứa dữ liệu test case, được lưu trữ trên dịch vụ S3 bucket của AWS.

Bảng `problem_points` định nghĩa cấu trúc tính điểm với các trường `based_points`, `difficulty_points`, `solve_rate`, `dynamic_points`, `contest_multiplier`, `first_solve_bonus` và `speed_bonus`, cho phép tính điểm linh hoạt theo nhiều tiêu chí.

Bảng `tags` lưu danh sách nhãn phân loại bài toán theo chủ đề thuật toán với các trường `name`, `slug` và `is_active`. Bảng liên kết `problems_tags` quản lý quan hệ nhiều-nhiều giữa bài toán và nhãn qua hai khóa ngoại `problem_id` và `tag_id`.

Bảng `users_problems` theo dõi lịch sử giải bài của từng người dùng với các trường `points_earned`, `first_attempt_date` và `solved_date`, cung cấp dữ liệu để tính thống kê và hiển thị trạng thái đã giải hay chưa cho từng bài toán.

3.4.4 Nhóm bảng quản lý cuộc thi

Nhóm bảng này xử lý toàn bộ vòng đời của một cuộc thi, từ khi tạo đến khi kết thúc và công bố kết quả.

Bảng `contests` lưu cấu hình cuộc thi với các trường `name`, `description`, `start_time`, `end_time`, `is_public`, `creator_id`, `slug`, `max_participant`, `registration_start`, `registration_end`, `is_rated` và `contest_style`. Trường `contest_style` cho phép cấu hình các kiểu thi khác nhau như ICPC hay IOI, ảnh hưởng trực tiếp đến cách tính điểm và xếp hạng.

Bảng liên kết `contests_problems` quản lý danh sách bài toán trong một cuộc thi với các trường `contest_id`, `problem_id`, `problem_order` và `points`, cho phép mỗi bài toán có trọng số điểm riêng trong từng cuộc thi.

Bảng `contest_registration` ghi nhận thông tin đăng ký tham gia cuộc thi của người dùng với các trường `user_id`, `contest_id`, `register_at` và `status`, phân biệt các trạng thái: đã đăng ký, đang chờ duyệt, hoặc bị từ chối.

Bảng `contest_participations` theo dõi kết quả của từng thí sinh trong cuộc thi với các trường `rank`, `score`, `penalty_time`, `problems_solved`, `points_earns`, `contest_id`, `user_id`, `type`, `rating_before` và `rating_after`. Đây là bảng chính phục vụ bảng xếp hạng và tính toán thay đổi rating sau mỗi kỳ thi.

Bảng `leaderboard_cache` lưu dữ liệu bảng xếp hạng đã được tính toán sẵn với các trường `type`, `rankings`, `total_users` và `last_updated`, giúp tăng tốc độ truy vấn và giảm tải cho cơ sở dữ liệu khi nhiều người dùng xem bảng xếp hạng đồng thời.

3.4.5 Nhóm bảng quản lý bài nộp

Bảng `submissions` là trung tâm của luồng chấm bài, lưu toàn bộ thông tin về mỗi lần nộp. Các trường dữ liệu bao gồm `source_code`, `memory_used`, `execution_time`, `submission_verdict`, `submission_status`, `submission_language`, `submission_date`, `problem_id`, `user_id`, `contest_id`, `test_cases_passed`, `total_test_cases`, `error_message` và `is_public`. Trường `submission_verdict` lưu kết quả cuối cùng (AC, WA, TLE, MLE, RE hoặc CE), còn `test_cases_passed` cung cấp thông tin chi tiết về số test case đã vượt qua, giúp người dùng định vị lỗi trong giải pháp của mình.

CHƯƠNG 4. KẾT QUẢ ĐÃ THỰC HIỆN

Tính đến thời điểm báo cáo giữa kỳ, dự án đã hoàn thành một khối lượng đáng kể công việc thực tế trên cả hai phần backend và frontend.

4.1 Backend – Java Spring Boot

Phần backend được triển khai bằng Java với Spring Boot. Kiến trúc tuân theo mô hình MVC kết hợp RESTful API, trong đó mỗi tài nguyên được ánh xạ thành một endpoint HTTP tương ứng với các phương thức GET, POST, PUT và DELETE.

4.1.1 Quản lý người dùng

Module quản lý người dùng là một trong những thành phần đầu tiên được hoàn thiện. Hệ thống hiện thực đầy đủ các chức năng xem danh sách người dùng, xem chi tiết từng tài khoản, cập nhật thông tin và thay đổi trạng thái hoạt động thông qua cơ chế kích hoạt và vô hiệu hóa – tương tác trực tiếp với các bảng `users`, `roles` và `users_roles`. Tất cả thao tác được bảo vệ bởi JWT, đảm bảo chỉ Admin mới có thể thực thi các thao tác quản trị tài khoản.

4.1.2 Quản lý bài toán

Module quản lý bài toán cung cấp đầy đủ API phục vụ cả Contest Creator và User thông thường, tương tác với các bảng `problems`, `test_cases`, `problem_points` và `problems_tags`. Phía quản trị, hệ thống hỗ trợ tạo mới, chỉnh sửa và xóa bài toán cùng các thông số cấu hình như giới hạn thời gian, giới hạn bộ nhớ, mức độ khó và bộ test cases. Phía người dùng, hệ thống hiện thực chức năng truy xuất chi tiết bài toán qua slug, tạo URL thân thiện và phù hợp với tiêu chuẩn RESTful hiện đại.

4.1.3 Trực quan hóa thuật toán

Tính năng trực quan hóa thuật toán đang được hoàn thiện. Sau khi có một bài nộp, người dùng có thể dùng chức năng này để trực quan hóa các biến và khối lệnh trong thuật toán bằng cách chạy các phần tử này theo từng bước (step). Tuy vậy, chức

năng này vẫn chưa hoàn thiện và cần được điều chỉnh để phù hợp với nhu cầu sử dụng của sinh viên Khoa Công nghệ Thông tin.

4.1.4 Xử lý bài nộp

Luồng xử lý bài nộp là tính năng trọng tâm và đã được tích hợp đầy đủ với Judge0. Khi người dùng nộp bài, backend tiếp nhận mã nguồn và ngôn ngữ lập trình, chuyển tiếp đến Judge0 qua API. Judge0 biên dịch và thực thi mã trong môi trường sandbox cô lập với các giới hạn lấy từ bảng `test_cases`, rồi trả về verdict tổng thể. Toàn bộ kết quả – verdict, thời gian thực thi, bộ nhớ sử dụng và số test case đã vượt qua – được lưu vào bảng `submissions` và trả về frontend. Luồng xử lý được thiết kế theo mô hình bất đồng bộ, đảm bảo server không bị chặn trong khi chờ Judge0 hoàn thành.

4.2 Tính năng hỗ trợ học tập bằng LLM

Một điểm nổi bật của TDTUOJ là tích hợp LLM để cung cấp ba tính năng hỗ trợ học tập thông minh, tương ứng với ba use case mở rộng từ Submit solution trong sơ đồ Use Case. Các tính năng này đã được hiện thực ở cả phía backend và frontend.

4.2.1 Phân tích độ phức tạp code

Sau khi bài nộp nhận kết quả AC, hệ thống có thể gửi mã nguồn đến LLM để nhận về phân tích độ phức tạp thời gian và không gian của giải thuật. Tính năng này giúp sinh viên không chỉ biết bài đúng hay sai mà còn hiểu được hiệu suất thực sự của giải pháp, từ đó có hướng tối ưu nếu cần.

4.2.2 Giải thích code và lỗi

Người dùng có thể yêu cầu LLM giải thích chi tiết đoạn mã đã viết hoặc phân tích nguyên nhân lỗi trong các lần nộp không đạt. Hệ thống gửi mã nguồn cùng thông tin bài toán và thông báo lỗi đến LLM, nhận về phần giải thích bao gồm mô tả logic tổng thể, phân tích các phần quan trọng và các điểm có thể cải thiện.

4.2.3 Gợi ý hướng giải quyết bài toán

Khi gặp khó khăn, người dùng có thể yêu cầu gợi ý mà không cần tiết lộ đáp án hoàn chỉnh. Hệ thống gửi thông tin bài toán đến LLM và nhận về gợi ý có cấu trúc – hướng dẫn người dùng suy nghĩ đúng hướng thông qua các câu hỏi dẫn dắt hoặc gợi ý về thuật toán phù hợp, giúp sinh viên phát triển tư duy giải quyết vấn đề một cách chủ động.

4.3 Frontend – React

Phần frontend được xây dựng bằng React. Giao diện được tổ chức thành các component có khả năng tái sử dụng cao, mỗi tính năng lớn được đóng gói thành một module riêng biệt.

4.3.1 Giao diện xem bài toán

Giao diện xem chi tiết bài toán đã được hoàn thiện với khả năng tải và hiển thị thông tin bài toán dựa trên slug truyền vào qua URL. Nội dung hiển thị bao gồm tên bài toán, mô tả, ràng buộc đầu vào đầu ra, giới hạn thời gian và bộ nhớ, mức độ khó, danh sách nhãn thuật toán và các ví dụ test case mẫu – lấy từ các bảng problems, test_cases và problems_tags qua backend API.

4.3.2 Giao diện quản lý người dùng

Giao diện quản lý người dùng dành cho Admin cung cấp đầy đủ các chức năng hiển thị danh sách tài khoản, xem chi tiết, chỉnh sửa thông tin và vô hiệu hóa tài khoản. Các thay đổi trên giao diện lập tức gọi API backend tương ứng và cập nhật trạng thái hiển thị theo kết quả phản hồi.

4.3.3 Tính năng nộp bài

Người dùng có thể nhập mã nguồn trực tiếp vào trình soạn thảo trên giao diện web, chọn ngôn ngữ lập trình và gửi bài nộp. Frontend xử lý toàn bộ luồng từ thu thập dữ liệu, gửi yêu cầu đến backend, chờ kết quả từ Judge0 và hiển thị verdict cuối cùng cùng thông tin chi tiết về số test case đạt, thời gian thực thi và bộ nhớ sử dụng.

CHƯƠNG 5. HƯỚNG PHÁT TRIỂN

5.1 Các tính năng còn lại cần hoàn thiện

Mặc dù phần lớn chức năng cốt lõi đã được hiện thực, vẫn còn một số tính năng quan trọng cần hoàn thiện để hệ thống đạt đủ năng lực vận hành thực tế. Bảng xếp hạng theo thời gian thực trong cuộc thi cần được phát triển để cập nhật liên tục thứ hạng người tham gia, sử dụng dữ liệu từ bảng `contest_participations` và `leaderboard_cache`. Tính năng phân tích độ phức tạp thuật toán qua AI Analyzer và nhập đề bài tự động từ PDF qua AI Parser Service cũng là các hạng mục còn lại cần hoàn thiện trong giai đoạn tiếp theo.

5.2 Kế hoạch hoàn thiện

5.2.1 Hoàn thiện chức năng cuộc thi và tổ chức

Ưu tiên hàng đầu là hoàn thiện toàn bộ vòng đời quản lý cuộc thi trên giao diện frontend. Hiện tại các API backend cho cuộc thi đã sẵn sàng, nhưng phần giao diện dành cho Contest Creator vẫn cần được phát triển để tạo mới, chỉnh sửa, giám sát tiến trình và công bố kết quả một cách trực quan. Tính năng quản lý tổ chức cũng sẽ được hoàn thiện, cho phép Admin tạo và quản lý các tổ chức qua bảng `organizations` và `users_organizations`, tạo ra cấu trúc phân cấp phù hợp với mô hình vận hành của một trường đại học. Ngoài ra, nhóm sẽ nghiên cứu ứng dụng bộ nhớ cache Redis để thực hiện việc xếp hạng người thi contest theo thời gian thực bằng điểm và thời gian giải được bài toán của người thi.

5.2.2 Trực quan hóa thuật toán

Như đã đề cập ở trên, nhóm sẽ hoàn thiện tính năng trực quan hóa thuật toán, cho phép sinh viên quan sát từng bước thực thi của các thuật toán phổ biến như sắp xếp, tìm kiếm, duyệt đồ thị và quy hoạch động thông qua hoạt ảnh minh họa trực tiếp. Người dùng có thể chọn thuật toán, nhập dữ liệu đầu vào tùy ý và điều chỉnh các bước

hiển thị để theo dõi quá trình thực thi từng bước chi tiết của một thuật toán. Đây là công cụ học tập đặc biệt hữu ích cho sinh viên đang tiếp cận các khái niệm thuật toán lần đầu.

5.2.3 Tích hợp trình gỡ lỗi

Hệ thống sẽ tích hợp một trình gỡ lỗi trực tuyến vào giao diện nộp bài, cho phép người dùng đặt breakpoint, theo dõi giá trị biến tại từng bước thực thi và quan sát trạng thái chương trình ngay trên trình duyệt. Tính năng này giúp sinh viên tự chẩn đoán lỗi logic một cách chủ động, đồng thời rèn luyện kỹ năng debugging – một kỹ năng thiết yếu trong nghề phát triển phần mềm. Tính năng này tương ứng với use case Visualize and debug code trong sơ đồ Use Case.

5.2.4 Tích hợp AI và hoàn thiện các dịch vụ

Sau khi hoàn thiện các chức năng quản lý cốt lõi, hệ thống sẽ tích hợp đầy đủ AI Analyzer vào luồng xử lý bài nộp để tự động phân tích và trả về độ phức tạp thuật toán ngay sau khi bài được chấp nhận, cùng với AI Parser Service để hỗ trợ giảng viên chuyển đổi đề bài từ PDF một cách tự động. Ngoài ra, nhóm sẽ nghiên cứu ứng dụng các hàng đợi như RabbitMQ để xử lý bất đồng bộ các tác vụ chấm bài của Judge0, nhằm giảm tải cho hệ thống và đảm bảo việc chấm bài diễn ra mượt mà và ổn định.

5.2.5 Hoàn thiện và đồng nhất giao diện người dùng

Hiện tại, giao diện người dùng đã có những phần cơ bản nhưng vẫn còn thiếu sự đồng nhất về thiết kế và trải nghiệm. Nhóm sẽ hoàn thiện toàn bộ giao diện, đảm bảo tính nhất quán về màu sắc, kiểu chữ, bố cục và tương tác trên tất cả các trang. Việc này không chỉ nâng cao thẩm mỹ mà còn cải thiện trải nghiệm người dùng, giúp sinh viên dễ dàng tiếp cận và sử dụng hệ thống một cách hiệu quả.

5.2.6 Kiểm thử và triển khai

Giai đoạn cuối tập trung vào kiểm thử toàn diện và triển khai hệ thống lên môi trường production.

Về kiểm thử, nhóm sẽ thực hiện unit test cho từng module backend để đảm bảo tính đúng đắn của từng thành phần riêng lẻ, sử dụng JUnit 5 kết hợp Mockito để kiểm thử các lớp Service độc lập với cơ sở dữ liệu thực. Song song đó, nhóm sẽ tiến hành integration test để xác minh luồng hoàn chỉnh từ Controller qua Service đến Repository, đảm bảo các thành phần phối hợp đúng với nhau. Cuối cùng, stress test sẽ được thực hiện để đánh giá khả năng chịu tải – đặc biệt ở các điểm nghẽn tiềm năng như luồng nộp bài đồng thời và truy vấn bảng xếp hạng trong thời gian diễn ra cuộc thi – sử dụng công cụ Apache JMeter hoặc k6 để mô phỏng hàng trăm người dùng hoạt động cùng lúc.

Về giám sát hệ thống, sau khi triển khai nhóm sẽ tích hợp bộ công cụ quan sát **Prometheus** và **Grafana** để theo dõi sức khỏe hệ thống theo thời gian thực. Prometheus sẽ chủ động scrape metrics từ bốn nguồn: Spring Boot backend thông qua endpoint `/actuator/prometheus` do thư viện Micrometer cung cấp, Redis thông qua `redis_exporter`, PostgreSQL thông qua `postgres_exporter`, và máy chủ vật lý thông qua `node_exporter`. Các metrics quan trọng cần theo dõi bao gồm tốc độ xử lý bài nộp, thời gian phản hồi của từng API endpoint, độ dài hàng đợi submission trong Redis, tỉ lệ cache hit của leaderboard, và mức sử dụng CPU/RAM trong các đợt thi cao điểm. Grafana kết nối với Prometheus làm data source và trực quan hóa toàn bộ các chỉ số trên thành dashboard tập trung, đồng thời cấu hình alert rules để tự động gửi cảnh báo qua email hoặc Slack khi các ngưỡng quan trọng bị vượt – chẳng hạn khi hàng đợi Redis tích lũy quá nhiều bài nộp chưa xử lý hoặc khi latency trung bình của Judge0 tăng đột biến.

Sau khi hệ thống vượt qua các ngưỡng kiểm thử, toàn bộ hạ tầng sẽ được triển khai lên **AWS**, tận dụng các dịch vụ phù hợp cho từng thành phần: EC2 cho backend và Judge0 workers, RDS cho PostgreSQL, ElastiCache cho Redis, và S3 cho lưu trữ file đề bài và test cases.

CHƯƠNG 6. KẾT LUẬN

6.1 Tổng kết

Đề tài xây dựng Hệ thống Chấm bài Trực tuyến TDTUOJ là một dự án có ý nghĩa thiết thực và phù hợp với nhu cầu thực tiễn của Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng. Tính đến thời điểm báo cáo giữa kỳ, dự án không chỉ dừng lại ở giai đoạn phân tích và thiết kế mà đã tiến vào triển khai thực tế với nhiều kết quả cụ thể và có thể kiểm chứng.

Về phân tích và thiết kế, hệ thống đã có sơ đồ Use Case đầy đủ với ba tác nhân người dùng có quan hệ kế thừa rõ ràng và ba tác nhân bên ngoài, cùng sơ đồ ERD hoàn chỉnh gồm 20 bảng phân thành bốn nhóm chức năng. Về triển khai, toàn bộ các module quản lý người dùng, bài toán, cuộc thi và bài nộp đã được hiện thực thành các RESTful API đầy đủ chức năng với bảo mật JWT, tích hợp Judge0 cho việc chấm bài. Phía frontend đã hoàn thiện giao diện xem bài toán theo slug, quản lý người dùng, tính năng nộp bài và ba tính năng hỗ trợ học tập thông minh bằng LLM. Toàn bộ mã nguồn được quản lý trên GitHub với 26 commit thể hiện tiến trình phát triển liên tục.

6.2 Ý nghĩa của đề tài

TDTUOJ không chỉ là một công cụ hỗ trợ giảng dạy đơn thuần mà là một nền tảng tổng thể có khả năng tạo ra tác động tích cực và lâu dài cho cộng đồng học thuật của nhà trường. Với sinh viên, hệ thống tạo ra môi trường luyện tập lập trình chuyên nghiệp với phản hồi tức thì, phân tích thuật toán chuyên sâu và hỗ trợ học tập thông minh từ LLM. Với giảng viên, hệ thống giải phóng nguồn lực khỏi các công việc lặp đi lặp lại và nâng cao hiệu quả tổ chức thi cử. Ở tầm nhìn rộng hơn, TDTUOJ góp phần xây dựng văn hóa lập trình thi đấu tại Trường Đại học Tôn Đức Thắng (TDTU), đồng thời thể hiện năng lực nghiên cứu và phát triển phần mềm thực tiễn của sinh viên Khoa Công nghệ Thông tin (CNTT).

TÀI LIỆU THAM KHẢO

- Amazon Web Services. (2024). Amazon s3 – object storage built to retrieve any amount of data [Truy cập ngày 01/03/2026]. <https://aws.amazon.com/s3>
- Docker Inc. (2024). Docker documentation [Truy cập ngày 01/03/2026]. <https://docs.docker.com>
- Došilović, H. Z. (2024). Judge0: Open-source online judge system [Truy cập ngày 01/03/2026]. <https://judge0.com>
- Grafana Labs. (2024). Grafana documentation [Truy cập ngày 01/03/2026]. <https://grafana.com/docs>
- Jones, M., Bradley, J., & Sakimura, N. (2015). RFC 7519: Json web token (JWT) [Truy cập ngày 01/03/2026]. <https://datatracker.ietf.org/doc/html/rfc7519>
- Meta Platforms, Inc. (2024). React – a javascript library for building user interfaces [Truy cập ngày 01/03/2026]. <https://react.dev>
- Redis Ltd. (2024). Redis documentation [Truy cập ngày 01/03/2026]. <https://redis.io/docs>
- The PostgreSQL Global Development Group. (2024). Postgresql 16 documentation [Truy cập ngày 01/03/2026]. <https://www.postgresql.org/docs/16/>
- The Prometheus Authors. (2024). Prometheus – monitoring system and time series database [Truy cập ngày 01/03/2026]. <https://prometheus.io/docs>
- Walls, C. (2016). *Spring boot in action*. Manning Publications.
- Wasik, S., Antczak, M., Badura, J., Laskowski, A., & Sternal, T. (2018). A survey on online judge systems and their applications. *ACM Computing Surveys*, 51(1), 1–34.